

Lecture 3: Numerical Optimization, Root Finding, and Fixed Points

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

Today's goals

1. Survey numerical optimization techniques and how to apply them in Julia
2. Learn root-finding methods and when to use them
3. Understand fixed-point iteration, which underlies much of computational economics

Optimization

- Suppose you want to find the minimum or maximum of a function, possibly subject to constraints:

$$\min_x f(x) \quad \text{s.t.} \quad g(x) \geq 0$$

- Note: maximizing f is equivalent to minimizing $-f$, so minimization techniques apply to maximization problems too
 - (Optimizers in Julia *minimize* by convention — remember the minus sign!)

When do we need to do this?

- Agent's choice problem:
 - Consumer maximizes utility subject to constraints
 - Firm chooses inputs/prices to maximize profits
- Estimation:
 - Ordinary Least Squares (OLS)
 - Maximum Likelihood Estimation (MLE): choose parameters which maximize the likelihood of the observed data
 - Simulated Method of Moments (SMM): choose parameters to minimize the distance between observed and simulated moments

How do we do this??

- Pencil and paper: take derivatives, use the Lagrangian, ...
 - This is what you did in first year — not always possible
- Grid search: guess and check
 - Evaluate the function at many points, keep the best
 - Accurate solutions require a very fine grid
 - Slow in many dimensions (10 parameters, 10 values each $\implies$ 10 billion combinations)
- Optimization algorithms (“smart” guess and check):
 - Use limited information about the function (such as derivatives) and previous guesses to decide the next guess
 - Lots of choices here; no perfect algorithm

Univariate optimization on an interval

- We often know the solution lives in $[a, b]$:
 - Search intensity normalized between 0 and 1
 - Hours of work between 0 and 80
 - Savings between 0 and net worth
- Common algorithm: **Brent's method**
 - Does not require derivatives
 - Combines bisection, secants, and inverse quadratic interpolation

```
using Optim
res = optimize(c -> -utility(c), 0.0, budget) # Brent is the default
Optim.minimizer(res)
```

- Optimization in multiple dimensions can be challenging
 - Curse of dimensionality
- If the function is twice differentiable with analytic gradient/Hessian, you can use **Newton's method**

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

- Derivatives tell you about the function's local behavior, so the algorithm finds good parameter values quickly
- In economics, you will rarely be dealing with such a nice function

Quasi-Newton methods and differentiation

- Quasi-Newton methods (most common: **BFGS** / **L-BFGS**) approximate the gradient/Hessian and apply a Newton-style updating rule
- Where do the derivatives come from?
 - **Finite differences**: $f'(x) \approx \frac{f(x+\delta) - f(x-\delta)}{2\delta}$; simple, but noisy and the choice of δ is non-trivial
 - **Automatic differentiation (AD)**: the computer applies the chain rule to your *code* and returns exact derivatives — fast and accurate, and easier than you'd think in Julia

```
using Optim
optimize(f, x0, LBFGS(); autodiff = :forward) # ForwardDiff-based AD
```

- AD requires your function to be written in pure, generic Julia (no non-differentiable steps like simulation with discrete choices) — when it applies, use it

Derivative-free methods

- Most common method: **Nelder-Mead** (the default in Optim.jl for multivariate problems)
- Intuition: “rolling a simplex down a hill” — keep a set of candidate points, reflect/expand/contract the worst one, repeat until the simplex shrinks
- Easy to use, no derivatives needed — but:
 - Doesn't use local information about function behavior
 - Can be easily confused by local minima
 - Can get stuck in flat areas or go around in circles
 - Convergence is slow
- Good default when your objective is non-smooth (e.g. simulated moments with a small number of draws)

Randomization and global methods

- Randomization helps when we have many local minima or non-smooth objectives
- Multistart / basin hopping:
 - Run a local algorithm (e.g. Nelder-Mead) from an initial point
 - From the candidate solution, randomly “hop” to a new initial point and run again
- Laplace-type / simulated annealing-style estimators:
 - Randomly jump around the parameter space
 - Accept better guesses (and sometimes worse ones, with some probability)
 - Jump size adapts based on the acceptance rate
- Cheap insurance in practice: always restart your optimizer from several starting points and check you get the same answer

- Many optimization algorithms stop after reaching a preset tolerance level
- Tighter tolerance improves the answer but increases run time
- Important with **nested** optimization, which is everywhere in economics:
 - Inner loop solves the model given a parameter guess (e.g. agents maximize utility)
 - Outer loop chooses parameters to fit the data (MLE/SMM/nested fixed point/BLP)
- Loose inner-loop tolerance $\implies$ noisy objective for the outer loop $\implies$ bad parameter guesses, failed convergence
- But overly tight tolerance everywhere can make estimation painfully slow — when is it good enough?

Root Finding

Root finding

- A **root** of f is an x such that $f(x) = 0$
- Where roots show up in economics:
 - Market clearing: find price p such that excess demand $Z(p) = 0$
 - First-order conditions: $u'(c) - \beta(1 + r) \mathbb{E}[u'(c')]$ (Euler equations)
 - Indifference/cutoff conditions in discrete choice and search models
- Root finding and optimization are cousins: an interior optimum is a root of the first-order condition
 - If you *have* the FOC, solving it is often faster and more accurate than optimizing

Bisection and Newton's method

- **Bisection**: if f is continuous with $f(a) < 0 < f(b)$, halve the interval, keep the half with the sign change, repeat
 - Slow but *bulletproof*: guaranteed convergence
- **Newton's method**: from guess x_n , follow the tangent line to zero:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- Very fast (quadratic convergence) near the solution
 - Can diverge from a bad starting point; needs f'
- **Secant/Brent**: Newton-like speed without derivatives; Brent's method combines these ideas with bisection safety

Root finding in Julia

```
using Roots
excess_demand(p) = demand(p) - supply(p)
p_star = find_zero(excess_demand, (0.1, 10.0)) # bracketing (Brent-like)
p_star = find_zero(excess_demand, 1.0)        # from a single guess

# Systems of equations: NonlinearSolve.jl
using NonlinearSolve
residuals(x, p) = [ ... ,      # equation 1 = 0
                  ... ]      # equation 2 = 0
prob = NonlinearProblem(residuals, x0)
sol = solve(prob)           # sol.u is the solution
```

- Prefer a **bracketing** method whenever you can bound the solution — it cannot fail

Fixed Points

- A **fixed point** of a mapping T is an x such that $T(x) = x$
- These are everywhere in economics:
 - The value function is the fixed point of the Bellman operator (Lecture 6!)
 - Stationary distributions of Markov chains
 - Equilibrium prices/beliefs: best responses map to themselves
- If T is a **contraction** (e.g. Bellman operator with discounting, by Blackwell's conditions), then iterating $x_{n+1} = T(x_n)$ converges to the unique fixed point from *any* starting guess

Fixed point iteration in practice

- Algorithm: guess x_0 , compute $x_{n+1} = T(x_n)$, stop when $\|x_{n+1} - x_n\| < \text{tol}$
- Convergence rate is governed by the modulus of contraction (β for Bellman operators):
 $\beta = 0.99$ means slow convergence
- If plain iteration is unstable or slow, use **dampening**:

$$x_{n+1} = \lambda T(x_n) + (1 - \lambda) x_n, \quad \lambda \in (0, 1]$$

- Fancier: Anderson acceleration (e.g. the fixed-point solvers in NonlinearSolve.jl) can dramatically cut iteration counts
- Note: a fixed point of T is a root of $f(x) = T(x) - x$, so all our root-finding machinery applies too

- Try to reduce dimensions, especially when getting started
- Plot your objective in 2D and 3D to understand its shape: Is it smooth? Does it have kinks? Multiple local minima?
- There is no perfect algorithm — try several, from several starting points
- Transformations of variables and scale can help with scale & bounds: $\log()$, $\exp()$, ...
 - e.g. optimize over $\log \sigma$ so that $\sigma > 0$ automatically
- To the code!